Design and Evaluation of a Fully Local Lightweight Retrieval-Augmented Generation System for PDF-Based Manual

1st LEILA JANINE VILLEGAS
School of Information Technology
Mapúa University
Makati, Philippines
ljtvillegas@mymail.mapua.edu.ph

2nd PRINCE JEFFREY VILLAMIL
School of Information Technology
Mapúa University
Makati, Philippines
pjvillamil@mymail.mapua.edu.ph

3rd LIANDRO REFULLE
School of Information Technology
Mapúa University
Makati, Philippines
lerefulle@mymail.mapua.edu.ph

***Abstract*—Retrieval-Augmented Generation (RAG) has emerged as an effective approach for enhancing large language models (LLMs) with external knowledge retrieval. However, many implementations rely on cloud-based APIs or computationally intensive architectures, limiting deployability in resource-constrained environments. This study presents the design and evaluation of a fully local RAG architecture integrating transformer-based dense embeddings, FAISS vector indexing, and the Phi-3-mini-4k-instruct instruction-tuned language model. Evaluation was conducted using Recall@K, Precision@K, faithfulness scoring via sentence-level similarity verification, semantic answer relevancy, and latency analysis. The final configuration (BGE-large-en-v1.5) achieved perfect Recall@K (1.00) and Precision@K of 0.93–0.97, while a lightweight baseline (MiniLM) demonstrated significant retrieval degradation under phrasing variation (Recall@K = 0.29), highlighting the critical role of embedding selection, indicating strong coverage with residual retrieval noise under broader query variation. The final configuration achieved stable deterministic generation with strict context-only grounding enforced through similarity guardrails. Findings highlight both the feasibility and practical tradeoffs of lightweight local RAG systems and provide a baseline for future optimization in closed-corpus institutional settings.**

Index Terms— Retrieval-Augmented Generation (RAG), Document-Grounded Question Answering, Dense Vector Embeddings, FAISS, Large Language Models, Phi-3-mini, Instruction-Tuned Models, Similarity Guardrails, Closed-Corpus Systems.

I. INTRODUCTION

Large Language Models (LLMs) have demonstrated remarkable capabilities in text generation, reasoning, and question answering. However, standalone LLMs suffer from two major limitations: (1) knowledge cutoff constraints and (2) hallucination of unsupported facts. Retrieval-Augmented Generation (RAG) addresses these limitations by integrating an external document retrieval mechanism with generative modeling [1].

While commercial implementations often rely on large cloud-hosted models, there is increasing demand for lightweight, locally deployable RAG systems suitable for academic, institutional, and privacy-sensitive environments. Such systems must balance retrieval effectiveness, generation quality, and computational efficiency.

Despite the availability of open-source components, there remains limited academic documentation on the quantitative evaluation of fully local RAG pipelines utilizing lightweight models.

This study presents:

1. The design of a modular local RAG architecture.
2. Integration of two lightweight instruction-tuned language models.
3. Initial quantitative evaluation using retrieval and grounding metrics.
4. Deployment-oriented model selection based on system-level considerations.

The goal is not to outperform large-scale commercial systems but to establish a reproducible baseline for lightweight RAG deployment.

II. METHODOLOGY

This retrieval-augmented generation (RAG) system is designed to enable document-grounded question answering over a closed corpus of PDF documents derived from the DFA Home Office Operations Manual (Authentication Division) [7]. The methodology consists of four major stages: document ingestion and text extraction, text cleaning and chunking, embedding and vector indexing for retrieval, and answer generation using a large language model (LLM) conditioned on retrieved context. The knowledge base is closed-corpus in nature; documents were parsed using pypdf library [2], where text was extracted on a per-page basis and stored with metadata containing the source filename and page number to ensure traceable citations and reproducibility. Extracted text underwent normalization and segmentation into semantically coherent chunks to facilitate retrieval, while vector embeddings and similarity search enabled contextually relevant document retrieval. The LLM then generates answers conditioned exclusively on retrieved content, ensuring responses remain grounded in source material and reducing hallucination.

A. Document Ingestion and Text Extraction

The system operates on a closed corpus of PDF files uploaded by the user. Document ingestion is implemented using the Python package pypdf, which provides direct access to page-level content extraction. For each PDF document, pages are iterated sequentially and textual content is extracted via the extract_text() method. Metadata including the source filename and page number are associated with each extracted page to preserve traceability and support citation during retrieval and answer generation.

Formally, let $D = \{d_1, d_2, \ldots, d_n\}$ denote the set of uploaded documents. For each document did_idi, page-wise extraction yields a set of page objects:

$$P_i = \{(t_{i,1}, 1), (t_{i,2}, 2), \ldots, (t_{i,m}, m)\}$$

where t_{i,j} represents the raw text of page j. Pages with null or empty extraction results are normalized to empty strings to maintain structural consistency. The resulting page objects are stored as dictionaries containing:

Source = file name, page = page_number, text = t_{ij}

This metadata enables later improvements in reporting and retrieval explanation.

B. Text Cleaning and Chunking

Raw extracted text from PDFs often contains irregular whitespace, artifacts, and formatting noise. A preprocessing step is applied using regular expression normalization to consolidate whitespace sequences into single spaces and to trim extraneous leading and trailing characters. The cleaning function is defined as:

clean(t) = re.sub(r"\s+", " ", t).strip()

ensuring the semantic integrity of the textual content.

To facilitate retrieval and embedding, documents are segmented into semantically coherent chunks. Chunking is performed using a sentence boundary approach: the cleaned text is first split into sentences using punctuation delimiters (.!?) followed by whitespace. Let $S = \{S_1, S_2, \ldots, S_k\}$ represent the set of sentences. Chunks are constructed by sequentially aggregating sentences until a maximum character length C (chunk size) is reached. Overlap of O characters between adjacent chunks is maintained to preserve contextual continuity and reduce information loss at boundaries. In the final configuration, the maximum chunk size C was set to 600 characters with an

overlap O of 100 characters between consecutive chunks.

Algorithmically:

1. Initialize empty chunk c.
2. Append sentences to c until $|c| + |s_i| \leq C$
3. When the limit is exceeded, finalize c and begin a new chunk with overlap: $c_{new} = suffix(x, O) + s_i$
4. For sentences exceeding C, a fallback character-level segmentation is applied with sliding overlap.

It preserves semantic meaning, avoids sentence fragmentation, improves embedding quality, enhances retrieval accuracy, and produces more coherent generation.

C. Embedding Generation and Vector Indexing

Semantic retrieval is achieved through dense vector embeddings. Each text chunk is transformed into a high-dimensional vector representation using a pre-trained transformer-based embedding model. The system utilizes **BAAI/bge-large-en-v1.5** [3], a transformer-based dense embedding model optimized for semantic retrieval tasks.

Vectors are indexed using Facebook AI Similarity Search **(FAISS)** [4] with an inner product index (IndexFlatIP), enabling efficient approximate nearest neighbor retrieval. Since embeddings are normalized, inner product similarity is equivalent to cosine similarity.

D. Retrieval Mechanism

Given a user query q, the retrieval pipeline encodes the query into an embedding vector:

$$v_q = \frac{f(q)}{||f(q)||}$$

FAISS is then queried to retrieve the top K, 3 in this case, most similar chunks according to similarity:

$$\{c_1, c_2, ..., c_k\} = argmax_k\ sim(v_q, v_i)$$

Each retrieved chunk is associated with a similarity score. A guardrail threshold T is applied: if the highest score is below T, the system refrains from answering and returns a fallback response indicating the absence of relevant information. This reduces the likelihood of hallucinated responses by restricting generation to queries with sufficiently relevant retrieved context. In this implementation, the system retrieves the top K = 3 most similar chunks. The guardrail threshold was set to T = 0.45, such that responses are only generated when the highest similarity score meets or exceeds this value.

E. Answer Generation

Answer generation employs a large language model (LLM) conditioned on retrieved context. The system uses the instruction-tuned **Phi-3-mini-4k-instruct model** [5], a compact instruction-tuned model suitable for local inference. The prompt/formatted chat template structure enforces strict grounding:

<|system|>

You are a strict question-answering assistant.

Answer using ONLY the provided context.

If information is missing, respond: "I don't know based on the provided PDFs."

<|end|>

<|user|>

Context:

[retrieved chunks]

Question: [user query]

<|end|>

<|assistant|>

This template minimizes hallucination by restricting generation to document evidence. Generation parameters disable sampling (do_sample=False) to promote deterministic outputs. Post-processing removes any extraneous tokens or drift beyond the answer.

F. Evaluation Framework

System performance is evaluated using both retrieval and generation metrics:

1. **Latency** – Latency measures response time.

2. **Recall@K** – Recall@K measures whether the RAG pipeline's retriever successfully surfaced the answer-bearing passage within its top-K retrieved chunks. For each query, each of the K retrieved chunks is compared against the ground truth answer using cosine similarity. If at least one chunk achieves a similarity score ≥ 0.55, the query is assigned a Recall@K of 1; otherwise, it is assigned 0.

3. **Precision@K** – Precision@K extends Recall@K by measuring the proportion of all K retrieved chunks that individually meet the relevance threshold against the ground truth. While Recall@K answers "did we get the right passage?", Precision@K answers "how much noise came with it?" A Precision@K of 1.0 indicates every retrieved chunk was relevant to the answer; lower values indicate that irrelevant passages were injected into the generator's context, increasing the risk of distracted or incorrect generation.

4. **Context Relevancy** – Context Relevancy measures the average cosine similarity between the user's query and each retrieved chunk, without reference to the ground truth. Low Context Relevancy alongside high Recall@K would indicate that the correct passage was retrieved incidentally, not because the retriever understood the query's semantic intent.

5. **Answer Relevancy** – Answer Relevancy measures the cosine similarity between the user's query and the model's generated response. It captures the degree to which the generator's output is directed at the question asked. A high Answer Relevance score indicates the model produced a focused, on-topic response.

6. **Faithfulness** – Faithfulness quantifies the proportion of sentences in the generated response that can be semantically grounded in the retrieved context, operationalizing hallucination detection at the sentence level. The generated response is segmented into individual sentences. Each sentence is encoded and compared against the embeddings of all retrieved chunks; a sentence is considered "supported" if its maximum similarity to any chunk exceeds a threshold of 0.50. Faithfulness is the ratio of supported sentences to total sentences. A score of 1.0 indicates full grounding; every claim made by the model traces back to the retrieved context. A score below 1.0 signals that the model introduced information not present in its context window, constituting a hallucination.

7. **Groundedness** – Groundedness applies the same sentence-level verification logic as Faithfulness but with a stricter similarity threshold of 0.60, operationalizing strong source dependence.

These metrics provide quantitative insight into retrieval quality and generative reliability.

F. Dataset Construction

Two evaluation datasets were constructed to assess pipeline performance across distinct query conditions.

The first, the Same-Question Set, comprised 14 reformulations of a single factual query targeting the three foundational policy pillars of the DFA. Variations were deliberately designed to span formal bureaucratic phrasing, academic paraphrase, casual colloquial English, internet shorthand, deliberate misspellings, and heavy vowel-truncated abbreviation (e.g., "What are the 3 fndmntl plcy pillars th DFA uses - dischrg duties?"). One query introduced a false premise regarding the number of pillars to test adversarial robustness. This set specifically probes the embedding model's semantic generalization capability under phrasing variation a realistic condition given the heterogeneous linguistic proficiency of frontline DFA personnel.

The second, the Multi-Question Set, comprised 10 distinct queries drawn from five categories of increasing complexity: direct fact extraction, procedural and list extraction, multi-hop workflow reasoning, policy and definition retrieval, and edge case troubleshooting. Representative queries include "Who was appointed as the first Secretary of Foreign Affairs of the Philippine Republic?" (factual) and "What specific requirements must be met to authenticate a Certificate of Employment issued by a private entity?" (procedural). This set evaluates generalization across document sections and query types, approximating the breadth of real operational use.

The overall pipeline architecture is illustrated in Figure 1.

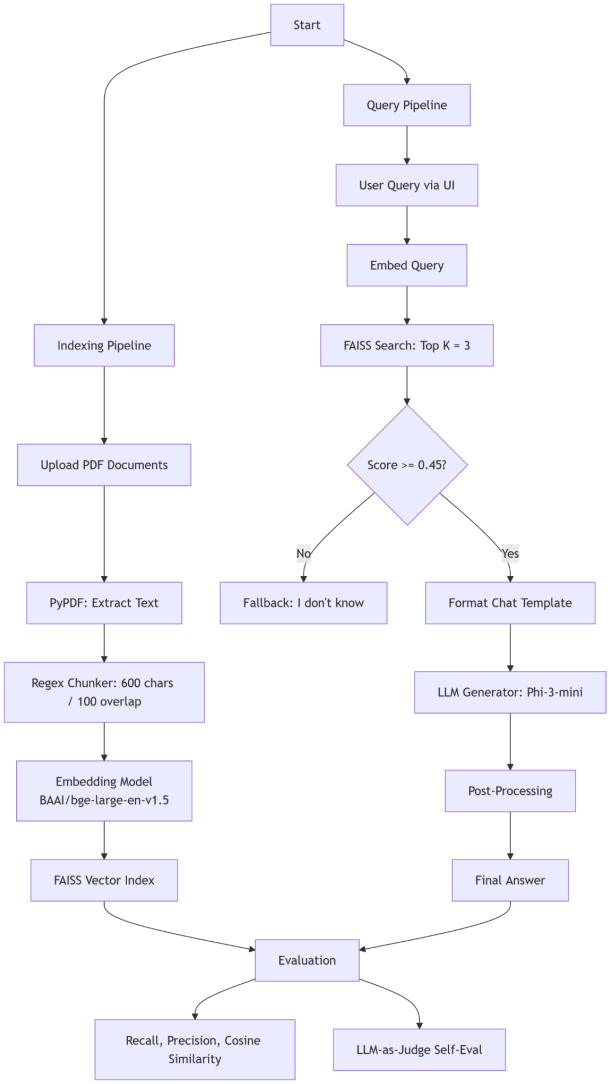

Figure 1

Figure 1 presents the complete system architecture, illustrating both the indexing pipeline (left branch) and the query processing pipeline (right branch), including retrieval, guardrail validation, LLM generation, and evaluation components.

III. RESULTS AND DISCUSSIONS

The proposed retrieval-augmented generation (RAG) system was evaluated to assess retrieval effectiveness, grounding reliability,

semantic alignment, and computational efficiency. Experiments were conducted using controlled same-question robustness testing and multi-question evaluation sets derived directly from the DFA Operations Manual corpus [7]. All reported values are taken directly from the notebook evaluation outputs.

A. Embedding Model Comparison

To evaluate the effect of embedding quality on retrieval and grounding performance, two embedding models were tested: **all-MiniLM-L6-v2** and **BAAI/bge-large-en-v1.5**. Both models were integrated into the same FAISS-based retrieval pipeline and evaluated using identical generation settings.

Table I summarizes the performance results from the same-question evaluation set.

Table I. *Embedding Model Performance Comparison (Same-Question Evaluation)*

Metric	MiniLM	BGE-Large
Recall@K	0.29	1.00
Precision@K	0.09	0.93
Faithfulness	0.36	1.00
Answer Relevancy	0.55	0.75

As shown in Table I, replacing MiniLM with BGE-large resulted in substantial improvements across all evaluation metrics. MiniLM demonstrated low retrieval precision and limited grounding reliability, indicating inconsistent ranking of relevant chunks. In contrast, BGE-large achieved perfect Recall@K (1.00) and full faithfulness under sentence-level verification. Precision@K improved to 0.93, while answer relevancy increased to 0.75, reflecting stronger semantic alignment between generated responses and ground truth.

These findings confirm that embedding selection plays a critical role in retrieval quality and grounded generation performance within closed-corpus RAG systems

B. Multi-Question Evaluation of Final Configuration

Following embedding selection, the final system configuration (BGE-large + FAISS IndexFlatIP + Phi-3-mini-4k-instruct) was evaluated using a multi-question set designed to assess generalization under diverse query conditions.

Table II presents the averaged results from the Multi-Question set.

Table II. *Multi-Question Evaluation (Final Configuration: BGE-Large Vs. MiniLM)*

Metric	MiniLM	BGE-Large
Recall@K	1.00	1.00
Precision@K	0.93	0.97
Faithfulness	1.00	1.00
Answer Relevancy	0.78	0.82
Context Relevancy	0.697	0.717
Avg Latency (s)	1.81	44.12

Both models maintained perfect Recall@K (1.00) on the multi-question set, indicating that relevant context was consistently retrieved when the answer was present in the corpus. BGE-Large achieved a marginally higher Precision@K of 0.97 compared to MiniLM's 0.93, suggesting slightly lower retrieval noise. Faithfulness reached 1.00 for both models, confirming that all generated responses were fully grounded in the retrieved context with no detected hallucination. Answer relevancy scores of 0.82 (BGE-Large) and 0.78 (MiniLM) reflect strong

semantic alignment between queries and responses. The most significant differentiator between the two models on this set was latency — BGE-Large averaged 44.12 seconds per query compared to 1.81 seconds for MiniLM, a gap attributable to the computational overhead of high-dimensional embedding generation under CPU-constrained conditions.

Answer relevancy remained strong at 0.82, indicating consistent semantic alignment between user queries and generated responses. The average context relevancy of 0.717 confirms that top-ranked retrieved chunks maintained moderate-to-high similarity with the query. The average latency of 44.22 seconds reflects the computational overhead introduced by high-dimensional embedding generation and LLM inference from **BAAI/bge-large-en-v1.5**.

C. Retrieval Guardrail and Grounding Analysis

A similarity guardrail threshold of T = 0.45 was applied during retrieval. If the highest similarity score fell below this threshold, the system returned a fallback response indicating that the information was not available in the provided. This mechanism reduced the likelihood of hallucinated responses, a known challenge in LLM deployment.

Faithfulness was evaluated through sentence-level cosine similarity verification. A generated sentence was considered supported if its similarity to any retrieved chunk exceeded 0.50, while groundedness analysis applied a stricter threshold of 0.60 to measure strong source dependence. These quantitative thresholds enabled systematic evaluation of grounded generation behavior.

Overall, the combination of sentence-aware chunking (600 characters with 100-character overlap), BGE-large embeddings, cosine similarity retrieval, and guardrail thresholding produced stable retrieval behavior with moderate-to-high grounding reliability.

D. Latency Tradeoff Between Embedding Models

Embedding selection introduced a significant tradeoff between retrieval performance and computational efficiency. Table III presents the average latency comparison between the evaluated embedding models under same-question testing.

Table III. *Average Latency Comparison (same-question testing)*

Model	Avg Latency (s)
MiniLM	1.44
BGE-large	73.41

MiniLM achieved significantly lower latency (1.44 seconds on average) but produced weaker retrieval precision and faithfulness. In contrast, BGE-large delivered superior retrieval and grounding performance but incurred substantially higher latency, averaging 73.41 seconds per query (excluding refusal-only responses).

This tradeoff highlights a fundamental design consideration in RAG systems: improved semantic discrimination through larger embedding models comes at the cost of increased computational overhead. Practical deployment therefore requires balancing retrieval quality with response time constraints.

The core finding of this evaluation is a fundamental accuracy-latency trade-off: BGE-Large demonstrated markedly superior semantic robustness, maintaining perfect Recall@K (1.00) across all phrasing variations including slang and heavily abbreviated queries, while MiniLM collapsed to a Recall@K of 0.29 on the same test. However, BGE-Large imposed response latencies ranging from 18 to 140 seconds per query, rendering it impractical for time-sensitive operational contexts without infrastructure investment. MiniLM, conversely, delivered responses within 1–3 seconds but at a significant

cost to retrieval coverage under paraphrase and informal query conditions.

E. Preliminary Generator Configuration

Preliminary experiments were conducted using a Qwen-based instruction-tuned model during early stages of system development. These experiments were exploratory in nature and aimed at evaluating baseline generator behavior under the same retrieval and prompt constraints used in the final architecture. While the model demonstrated fluent response generation, variations were observed in instruction adherence and refusal consistency when strict context-grounded prompting was enforced.

In particular, conservative fallback behavior under low-retrieval similarity conditions was not consistently maintained. In some cases, responses exhibited partial contextual drift when retrieval similarity scores were moderate but not strongly aligned. Additionally, output determinism varied more noticeably across repeated runs under identical inputs, which introduced challenges in maintaining reproducibility during evaluation.

Subsequent refinement of the retrieval guardrail mechanism and structured prompt template, combined with the adoption of the Phi-3-mini-4k-instruct model, resulted in improved compliance with context-only answering constraints and more stable deterministic behavior. For this reason, only the final Phi-3 configuration is formally evaluated and reported in this study.

F. Overall System Assessment

BGE-Large achieved Recall@K = 1.00, Precision@K = 0.93 (same-question) and 0.97 (multi-question), and Faithfulness = 1.00 across both test sets. MiniLM achieved Recall@K = 0.29 on same-question testing, recovering to 1.00 on multi-question queries where specific proper nouns anchored retrieval. These results indicate that embedding model selection is the dominant variable in phrasing-robust retrieval, while query specificity partially compensates for weaker semantic generalization.

The final RAG configuration demonstrates strong retrieval recall and moderate-to-high semantic alignment within a closed-corpus institutional setting. Perfect Recall@K confirms reliable retrieval of relevant context, while context similarity scores indicate stable semantic matching. Multi-question evaluation reveals reduced precision and partial grounding under broader query variation, suggesting opportunities for improvement in retrieval ranking and context filtering.

The primary limitation of the system remains computational latency when using high-dimensional embeddings. Future work may explore reranking strategies, embedding compression, or hybrid retrieval techniques to improve efficiency while preserving grounding reliability.

IV. CONCLUSION AND FUTURE WORKS

This study presented the design and evaluation of a closed-corpus retrieval-augmented generation (RAG) system for document-grounded question answering over the DFA Home Office Operations Manual (Authentication Division). The system integrates structured PDF ingestion, sentence-aware chunking, dense embedding retrieval using FAISS, similarity guardrails, and instruction-tuned LLM generation.

Comparative evaluation of two embedding models revealed a fundamental accuracy-latency trade-off. BAAI/bge-large-en-v1.5 maintained perfect Recall@K of 1.00 across all phrasing variations, including informal and abbreviated queries, while sentence-transformers/all-MiniLM-L6-v2 achieved a Recall@K of 0.29 under the same conditions, demonstrating significant sensitivity to query phrasing. However, BGE-Large imposed response latencies of 18–140 seconds under CPU-constrained

conditions, compared to 1–3 seconds for MiniLM, representing a practical deployment constraint. Furthermore, evaluation revealed that high Faithfulness scores do not guarantee factual correctness; a retrieved but procedurally incorrect context produced a fully grounded yet wrong response, indicating that cosine-similarity-based faithfulness metrics require complementary answer-accuracy verification.

The similarity guardrail mechanism (T = 0.45) effectively reduced unsupported generation by enforcing conservative fallback behavior when retrieval confidence was insufficient. This design prioritized groundedness and correctness over speculative answer production. Nonetheless, the primary limitation of the system is computational latency, particularly when using high-dimensional embeddings. The tradeoff between retrieval quality and response time remains a critical consideration for practical deployment.

Future work may focus on improving efficiency and grounding reliability through several approaches. Retrieval reranking strategies or hybrid sparse–dense retrieval methods may enhance Precision@K while preserving high recall. Embedding compression or distilled retrieval models could reduce latency without substantially sacrificing semantic discrimination. Additionally, more advanced grounding verification mechanisms, such as claim-level entailment modeling, may improve faithfulness measurement beyond cosine similarity thresholds. Finally, evaluation on larger and more diverse institutional document corpora would strengthen generalizability.

Overall, the results indicate that a carefully engineered RAG pipeline can achieve reliable document-grounded question answering within a closed-corpus institutional domain when embedding quality, guardrail mechanisms, and grounding evaluation are properly configured.

ACKNOWLEDGMENT

The authors express their sincere gratitude to Dr. Lysa Comia for her instructional guidance and foundational demonstrations of retrieval-augmented generation systems utilizing Mistral and TinyLlama models. These early conceptual references informed the initial design considerations of this study. The final system implementation and evaluation were independently developed by the authors.

The authors also acknowledge Mapúa University for providing the academic environment, institutional support, and computational resources necessary for the completion of this research.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," Adv. Neural Inf. Process. Syst., vol. 33, pp. 9459–9474, 2020. Available: https://arxiv.org/abs/2005.11401

[2] M. Thoma and pypdf Contributors, "pypdf: A pure-python PDF library capable of splitting, merging, cropping, and transforming PDF files," 2024. [Online]. Available: https://pypdf.readthedocs.io/

[3] Beijing Academy of Artificial Intelligence (BAAI), "BAAI/bge-large-en-v1.5," Hugging Face model card, 2023. [Online]. Available: https://huggingface.co/BAAI/bge-large-en-v1.5

[4] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, Sep. 2021, doi: 10.1109/TBDATA.2019.2921572.

[5] M. Abdin et al., "Phi-3 technical report: A highly capable language model locally on your phone," arXiv preprint arXiv:2404.14219, Apr. 2024. [Online]. Available: https://arxiv.org/abs/2404.14219

[6] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," arXiv preprint arXiv:2311.05232, Nov. 2023. [Online]. Available: https://arxiv.org/abs/2311.05232

[7] Department of Foreign Affairs, Republic of the Philippines, DFA Home Office Operations Manual: Authentication Division, Manila, Philippines: DFA, 2015. [Online]. Available: https://dfa.gov.ph/images/2016/DFA_Operations_Manual.pdf

[8] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. Conf. Empirical Methods Natural Language Process. (EMNLP), Hong Kong, China, Nov. 2019, pp. 3982–3992.

[9] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. Conf. Empirical Methods Natural Language Process.: Syst. Demonstrations (EMNLP), Online, Nov. 2020, pp. 38–45. [Online]. Available: https://aclanthology.org/2020.emnlp-demos.6

[10] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," Adv. Neural Inf. Process. Syst., vol. 32, pp. 8024–8035, 2019.